JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Properties

java.lang.Object
  java.util.Dictionary<Object,Object>
    java.util.Hashtable<Object,Object>
      java.util.Properties

All Implemented Interfaces:

Serializable, Cloneable, Map<Object,Object>

Direct Known Subclasses:

Provider

```
public class Properties
extends Hashtable<Object,Object>
```

The `Properties` class represents a persistent set of properties. The `Properties` can be saved to a stream or loaded from a stream. Each key and its corresponding value in the property list is a string.

A property list can contain another property list as its "defaults"; this second property list is searched if the property key is not found in the original property list.

Because `Properties` inherits from `Hashtable`, the `put` and `putAll` methods can be applied to a `Properties` object. Their use is strongly discouraged as they allow the caller to insert entries whose keys or values are not `Strings`. The `setProperty` method should be used instead. If the `store`, `save`, or `list` method is called on a "compromised" `Properties` object that contains a non-`String` key or value, the call will fail. Similarly, the call to the `propertyNames` method will fail if it is called on a "compromised" `Properties` object that contains a non-`String` key.

The iterators returned by the `iterator` method of this class's "collection views" (that is, `entrySet()`, `keySet()`, and `values()`) may not fail-fast (unlike the Hashtable implementation). These iterators are guaranteed to traverse elements as they existed upon construction exactly once, and may (but are not guaranteed to) reflect any modifications subsequent to construction.

The `load(Reader)` / `store(Writer, String)` methods load and store properties from and to a character based stream in a simple line-oriented format specified below. The `load(InputStream)` / `store(OutputStream, String)` methods work the same way as the load(Reader)/store(Writer, String) pair, except the input/output stream is encoded in ISO 8859-1 character encoding. Characters that cannot be directly represented in this encoding can be written using Unicode escapes as defined in section §3.3 of *The Java Language Specification*; only a single 'u' character is allowed in an escape sequence.

The `loadFromXML(InputStream)` and `storeToXML(OutputStream, String, String)` methods load and store properties in a simple XML format. By default the UTF-8 character encoding is used, however a specific encoding may be specified if required. Implementations are required to support UTF-8 and UTF-16 and may support other encodings. An XML properties document has the following DOCTYPE declaration:

```
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
```

Note that the system URI (http://java.sun.com/dtd/properties.dtd) is *not* accessed when exporting or importing properties; it merely serves as a string to uniquely identify the DTD, which is:

```
    <?xml version="1.0" encoding="UTF-8"?>

    <!-- DTD for properties -->

    <!ELEMENT properties ( comment?, entry* ) >

    <!ATTLIST properties version CDATA #FIXED "1.0">

    <!ELEMENT comment (#PCDATA) >

    <!ELEMENT entry (#PCDATA) >

    <!ATTLIST entry key CDATA #REQUIRED>
```

This class is thread-safe: multiple threads can share a single `Properties` object without the need for external synchronization.

API Note:

The `Properties` class does not inherit the concept of a load factor from its superclass, `Hashtable`.

Since:

1.0

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description

protected Properties	**defaults**	A property list that contains default values for any keys not found in this property list.

Constructor Summary

Constructors

Constructor	Description
Properties()	Creates an empty property list with no default values.
Properties(int initialCapacity)	Creates an empty property list with no default values, and with an initial size accommodating the specified number of elements without the need to dynamically resize.
Properties(Properties defaults)	Creates an empty property list with the specified defaults.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
String	**getProperty**(String key)	Searches for the property with the specified key in this property list.
String	**getProperty**(String key, String defaultValue)	Searches for the property with the specified key in this property list.
void	**list**(PrintStream out)	Prints this property list out to the specified output stream.
void	**list**(PrintWriter out)	Prints this property list out to the specified output stream.

void	**load**(InputStream inStream)	Reads a property list (key and element pairs) from the input byte stream.
void	**load**(Reader reader)	Reads a property list (key and element pairs) from the input character stream in a simple line-oriented format.
void	**loadFromXML**(InputStream in)	Loads all of the properties represented by the XML document on the specified input stream into this properties table.
Enumeration<?>	**propertyNames**()	Returns an enumeration of all the keys in this property list, including distinct keys in the default property list if a key of the same name has not already been found from the main properties list.
void	**save**(OutputStream out, String comments)	**Deprecated.** This method does not throw an IOException if an I/O error occurs while saving the property list.
Object	**setProperty**(String key, String value)	Calls the `Hashtable` method `put`.
void	**store**(OutputStream out, String comments)	Writes this property list (key and element pairs) in this `Properties` table to the output stream in a format suitable for

		loading into a `Properties` table using the `load(InputStream)` method.
`void`	**`store`**`(Writer writer, String comments)`	Writes this property list (key and element pairs) in this `Properties` table to the output character stream in a format suitable for using the `load(Reader)` method.
`void`	**`storeToXML`**`(OutputStream os, String comment)`	Emits an XML document representing all of the properties contained in this table.
`void`	**`storeToXML`**`(OutputStream os, String comment, String encoding)`	Emits an XML document representing all of the properties contained in this table, using the specified encoding.
`void`	**`storeToXML`**`(OutputStream os, String comment, Charset charset)`	Emits an XML document representing all of the properties contained in this table, using the specified encoding.
`Set<String>`	**`stringPropertyNames`**`()`	Returns an unmodifiable set of keys from this property list where the key and its corresponding value are strings, including distinct keys in the default property list if a key of the same name has not already been

	found from the main properties list.

Methods declared in class Hashtable

`clear`, `clone`, `compute`, `computeIfAbsent`, `computeIfPresent`, `contains`, `containsKey`, `containsValue`, `elements`, `entrySet`, `equals`, `get`, `hashCode`, `isEmpty`, `keys`, `keySet`, `merge`, `put`, `putAll`, `rehash`, `remove`, `size`, `toString`, `values`

Methods declared in class Object

`finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods declared in interface Map

`forEach`, `getOrDefault`, `putIfAbsent`, `remove`, `replace`, `replace`, `replaceAll`

Field Details

defaults

```
protected volatile Properties defaults
```

A property list that contains default values for any keys not found in this property list.

Constructor Details

Properties

```
public Properties()
```

Creates an empty property list with no default values.

Implementation Note:

The initial capacity of a `Properties` object created with this constructor is unspecified.

Properties

```
public Properties(int initialCapacity)
```

Creates an empty property list with no default values, and with an initial size accommodating the specified number of elements without the need to dynamically resize.

Parameters:

`initialCapacity` - the `Properties` will be sized to accommodate this many elements

Throws:

`IllegalArgumentException` - if the initial capacity is less than zero.

Since:

10

Properties

```
public Properties(Properties defaults)
```

Creates an empty property list with the specified defaults.

Implementation Note:

The initial capacity of a `Properties` object created with this constructor is unspecified.

Parameters:

`defaults` - the defaults.

Method Details

setProperty

```
public Object setProperty(String key,
                          String value)
```

Calls the `Hashtable` method `put`. Provided for parallelism with the `getProperty` method. Enforces use of strings for property keys and values. The value returned is the result of the `Hashtable` call to `put`.

Parameters:

`key` - the key to be placed into this property list.

`value` - the value corresponding to `key`.

Returns:

the previous value of the specified key in this property list, or `null` if it did not have one.

Since:

1.2

See Also:

`getProperty(String)`

load

```
public void load(Reader reader)
          throws IOException
```

Reads a property list (key and element pairs) from the input character stream in a simple line-oriented format.

Properties are processed in terms of lines. There are two kinds of lines, *natural lines* and *logical lines*. A natural line is defined as a line of characters that is terminated either by a set of line terminator characters (`\n` or `\r` or `\r\n`) or by the end of the stream. A natural line may be either a blank line, a comment line, or hold all or some of a key-element pair. A logical line holds all the data of a key-element pair, which may be spread out across several adjacent natural lines by escaping the line terminator sequence with a backslash character `\`. Note that a comment line cannot be extended in this manner; every natural line that is a comment must have its own comment indicator, as described below. Lines are read from input until the end of the stream is reached.

A natural line that contains only white space characters is considered blank and is ignored. A comment line has an ASCII `'#'` or `'!'` as its first non-whitespace character; comment lines are also ignored and do not encode key-element information. In addition to line terminators, this format considers the characters space (`' '`, `'\u0020'`), tab (`'\t'`, `'\u0009'`), and form feed (`'\f'`, `'\u000C'`) to be white space.

If a logical line is spread across several natural lines, the backslash escaping the line terminator sequence, the line terminator sequence, and any white space at the start of the following line have no effect on the key or element values. The remainder of the discussion of key and element parsing (when loading) will assume all the characters constituting the key and element appear on a single natural line after line continuation characters have been removed. Note that it is *not* sufficient to only examine the character preceding a line terminator sequence to decide if the line terminator is escaped; there must be an odd number of contiguous backslashes for the line

terminator to be escaped. Since the input is processed from left to right, a non-zero even number of 2*n* contiguous backslashes before a line terminator (or elsewhere) encodes *n* backslashes after escape processing.

The key contains all of the characters in the line starting with the first non-whitespace character and up to, but not including, the first unescaped `'='`, `':'`, or white space character other than a line terminator. All of these key termination characters may be included in the key by escaping them with a preceding backslash character; for example,

```
\:\=
```

would be the two-character key `":="`. Line terminator characters can be included using `\r` and `\n` escape sequences. Any white space after the key is skipped; if the first non-whitespace character after the key is `'='` or `':'`, then it is ignored and any white space characters after it are also skipped. All remaining characters on the line become part of the associated element string; if there are no remaining characters, the element is the empty string `""`. Once the raw character sequences constituting the key and element are identified, escape processing is performed as described above.

As an example, each of the following three lines specifies the key `"Truth"` and the associated element value `"Beauty"`:

```
Truth = Beauty
 Truth:Beauty
Truth                    :Beauty
```

As another example, the following three lines specify a single property:

```
fruits                           apple, banana, pear, \
                                 cantaloupe, watermelon, \
                                 kiwi, mango
```

The key is `"fruits"` and the associated element is:

```
"apple, banana, pear, cantaloupe, watermelon, kiwi, mango"
```

Note that a space appears before each `\` so that a space will appear after each comma in the final result; the `\`, line terminator, and leading white space on the continuation line are merely discarded and are *not* replaced by one or more other characters.

As a third example, the line:

```
cheeses
```

specifies that the key is `"cheeses"` and the associated element is the empty string `""`.

Characters in keys and elements can be represented in escape sequences similar to those used for character and string literals (see sections §3.3 and §3.10.6 of *The Java Language Specification*). The differences from the character escape sequences and Unicode escapes used for characters and strings are:

- Octal escapes are not recognized.
- The character sequence `\b` does *not* represent a backspace character.
- The method does not treat a backslash character, `\`, before a non-valid escape character as an error; the backslash is silently dropped. For example, in a Java string the sequence `"\z"` would cause a compile time error. In contrast, this method silently drops the backslash. Therefore, this method treats the two character sequence `"\b"` as equivalent to the single character `'b'`.
- Escapes are not necessary for single and double quotes; however, by the rule above, single and double quote characters preceded by a backslash still yield single and double quote characters, respectively.
- Only a single 'u' character is allowed in a Unicode escape sequence.

The specified stream remains open after this method returns.

Parameters:

`reader` - the input character stream.

Throws:

`IOException` - if an error occurred when reading from the input stream.

`IllegalArgumentException` - if a malformed Unicode escape appears in the input.

`NullPointerException` - if `reader` is null.

Since:

1.6

load

```
public void load(InputStream inStream)
          throws IOException
```

Reads a property list (key and element pairs) from the input byte stream. The input stream is in a simple line-oriented format as specified in `load(Reader)` and is assumed to use the ISO 8859-1 character encoding; that is each byte is

one Latin1 character. Characters not in Latin1, and certain special characters, are represented in keys and elements using Unicode escapes as defined in section §3.3 of *The Java Language Specification*.

The specified stream remains open after this method returns.

Parameters:

`inStream` - the input stream.

Throws:

`IOException` - if an error occurred when reading from the input stream.

`IllegalArgumentException` - if the input stream contains a malformed Unicode escape sequence.

`NullPointerException` - if `inStream` is null.

Since:

1.2

save

```
@Deprecated
public void save(OutputStream out,
                 String comments)
```

Deprecated.
This method does not throw an IOException if an I/O error occurs while saving the property list. The preferred way to save a properties list is via the `store(OutputStream out, String comments)` method or the `storeToXML(OutputStream os, String comment)` method.

Calls the `store(OutputStream out, String comments)` method and suppresses IOExceptions that were thrown.

Parameters:

`out` - an output stream.

`comments` - a description of the property list.

Throws:

`ClassCastException` - if this `Properties` object contains any keys or values that are not `Strings`.

store

```
public void store(Writer writer,
                  String comments)
           throws IOException
```

Writes this property list (key and element pairs) in this `Properties` table to the output character stream in a format suitable for using the `load(Reader)` method.

Properties from the defaults table of this `Properties` table (if any) are *not* written out by this method.

If the comments argument is not null, then an ASCII # character, the comments string, and a line separator are first written to the output stream. Thus, the `comments` can serve as an identifying comment. Any one of a line feed (`\n`), a carriage return (`\r`), or a carriage return followed immediately by a line feed (`\r\n`) in comments is replaced by a `line separator` and if the next character in comments is not character # or character `!` then an ASCII # is written out after that line separator.

If the `java.properties.date` is set on the command line and is non-empty (as determined by `String.isEmpty`), a comment line is written as follows. First, a # character is written, followed by the contents of the property, followed by a line separator. Any line terminator characters in the value of the system property are treated the same way as noted above for the comments argument. If the system property is not set or is empty, a comment line is written as follows. First, a # character is written, followed by the current date and time formatted as if by the `Date.toString` method, followed by a line separator.

Then every entry in this `Properties` table is written out, one per line. For each entry the key string is written, then an ASCII =, then the associated element string. For the key, all space characters are written with a preceding `\` character. For the element, leading space characters, but not embedded or trailing space characters, are written with a preceding `\` character. The key and element characters #, `!`, =, and `:` are written with a preceding backslash to ensure that they are properly loaded.

After the entries have been written, the output stream is flushed. The output stream remains open after this method returns.

Implementation Requirements:

The keys and elements are written in the natural sort order of the keys in the `entrySet()` unless `entrySet()` is overridden by a subclass to return a different value than `super.entrySet()`.

Parameters:

`writer` - an output character stream writer.

`comments` - a description of the property list.

Throws:

`IOException` - if writing this property list to the specified output stream throws an `IOException`.

`ClassCastException` - if this `Properties` object contains any keys or values that are not `Strings`.

`NullPointerException` - if `writer` is null.

Since:

1.6

store

```
public void store(OutputStream out,
                  String comments)
           throws IOException
```

Writes this property list (key and element pairs) in this `Properties` table to the output stream in a format suitable for loading into a `Properties` table using the `load(InputStream)` method.

Properties from the defaults table of this `Properties` table (if any) are *not* written out by this method.

This method outputs the comments, properties keys and values in the same format as specified in `store(Writer)`, with the following differences:

- The stream is written using the ISO 8859-1 character encoding.
- Characters not in Latin-1 in the comments are written as `\u`*xxxx* for their appropriate unicode hexadecimal value *xxxx*.
- Characters less than `\u0020` and characters greater than `\u007E` in property keys or values are written as `\u`*xxxx* for the appropriate hexadecimal value *xxxx*.

After the entries have been written, the output stream is flushed. The output stream remains open after this method returns.

Parameters:

`out` - an output stream.

`comments` - a description of the property list.

Throws:

`IOException` - if writing this property list to the specified output stream throws an `IOException`.

`ClassCastException` - if this `Properties` object contains any keys or values that are not `Strings`.

`NullPointerException` - if `out` is null.

Since:

1.2

loadFromXML

```
public void loadFromXML(InputStream in)
                 throws IOException,
                        InvalidPropertiesFormatException
```

Loads all of the properties represented by the XML document on the specified input stream into this properties table.

The XML document must have the following DOCTYPE declaration:

```
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
```

Furthermore, the document must satisfy the properties DTD described above.

An implementation is required to read XML documents that use the "UTF-8" or "UTF-16" encoding. An implementation may support additional encodings.

The specified stream is closed after this method returns.

Parameters:

in - the input stream from which to read the XML document.

Throws:

IOException - if reading from the specified input stream results in an IOException.

UnsupportedEncodingException - if the document's encoding declaration can be read and it specifies an encoding that is not supported

InvalidPropertiesFormatException - Data on input stream does not constitute a valid XML document with the mandated document type.

NullPointerException - if in is null.

Since:

1.5

External Specifications

Extensible Markup Language (XML) 1.0 (Fifth Edition)

See Also:

storeToXML(OutputStream, String, String),
Character Encoding in Entities

storeToXML

```
public void storeToXML(OutputStream os,
                       String comment)
                throws IOException
```

Emits an XML document representing all of the properties contained in this table.

An invocation of this method of the form `props.storeToXML(os, comment)` behaves in exactly the same way as the invocation `props.storeToXML(os, comment, "UTF-8");`.

Parameters:

`os` - the output stream on which to emit the XML document.

`comment` - a description of the property list, or `null` if no comment is desired.

Throws:

`IOException` - if writing to the specified output stream results in an `IOException`.

`NullPointerException` - if `os` is null.

`ClassCastException` - if this `Properties` object contains any keys or values that are not `Strings`.

Since:

1.5

See Also:

`loadFromXML(InputStream)`

storeToXML

```
public void storeToXML(OutputStream os,
                       String comment,
                       String encoding)
                throws IOException
```

Emits an XML document representing all of the properties contained in this table, using the specified encoding.

The XML document will have the following DOCTYPE declaration:

```
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
```

If the specified comment is `null` then no comment will be stored in the document.

An implementation is required to support writing of XML documents that use the "UTF-8" or "UTF-16" encoding. An implementation may support additional encodings.

The specified stream remains open after this method returns.

This method behaves the same as storeToXML(OutputStream os, String comment, Charset charset) except that it will look up the charset using the given encoding name.

Parameters:

os - the output stream on which to emit the XML document.

comment - a description of the property list, or null if no comment is desired.

encoding - the name of a supported character encoding

Throws:

IOException - if writing to the specified output stream results in an IOException.

UnsupportedEncodingException - if the encoding is not supported by the implementation.

NullPointerException - if os is null, or if encoding is null.

ClassCastException - if this Properties object contains any keys or values that are not Strings.

Since:

1.5

External Specifications

Extensible Markup Language (XML) 1.0 (Fifth Edition)

See Also:

loadFromXML(InputStream),
Character Encoding in Entities

storeToXML

```
public void storeToXML(OutputStream os,
                       String comment,
                       Charset charset)
                throws IOException
```

Emits an XML document representing all of the properties contained in this table, using the specified encoding.

The XML document will have the following DOCTYPE declaration:

```
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
```

If the specified comment is `null` then no comment will be stored in the document.

An implementation is required to support writing of XML documents that use the "`UTF-8`" or "`UTF-16`" encoding. An implementation may support additional encodings.

Unmappable characters for the specified charset will be encoded as numeric character references.

The specified stream remains open after this method returns.

Parameters:

`os` - the output stream on which to emit the XML document.

`comment` - a description of the property list, or `null` if no comment is desired.

`charset` - the charset

Throws:

`IOException` - if writing to the specified output stream results in an `IOException`.

`NullPointerException` - if `os` or `charset` is `null`.

`ClassCastException` - if this `Properties` object contains any keys or values that are not `Strings`.

Since:

10

External Specifications

Extensible Markup Language (XML) 1.0 (Fifth Edition)

See Also:

`loadFromXML(InputStream)`,
Character Encoding in Entities

getProperty

```
public String getProperty(String key)
```

Searches for the property with the specified key in this property list. If the key is not found in this property list, the default property list, and its defaults, recursively, are then checked. The method returns `null` if the property is not found.

Parameters:

key - the property key.

Returns:

the value in this property list with the specified key value.

See Also:

`setProperty(String, String)`,
`defaults`

getProperty

```
public String getProperty(String key,
                          String defaultValue)
```

Searches for the property with the specified key in this property list. If the key is not found in this property list, the default property list, and its defaults, recursively, are then checked. The method returns the default value argument if the property is not found.

Parameters:

key - the hashtable key.

defaultValue - a default value.

Returns:

the value in this property list with the specified key value.

See Also:

`setProperty(String, String)`,
`defaults`

propertyNames

```
public Enumeration<?> propertyNames()
```

Returns an enumeration of all the keys in this property list, including distinct keys in the default property list if a key of the same name has not already been found from the main properties list.

Returns:

an enumeration of all the keys in this property list, including the keys in the default property list.

Throws:

`ClassCastException` - if any key in this property list is not a string.

See Also:

`Enumeration`, `defaults`, `stringPropertyNames()`

stringPropertyNames

```
public Set<String> stringPropertyNames()
```

Returns an unmodifiable set of keys from this property list where the key and its corresponding value are strings, including distinct keys in the default property list if a key of the same name has not already been found from the main properties list. Properties whose key or value is not of type `String` are omitted.

The returned set is not backed by this `Properties` object. Changes to this `Properties` object are not reflected in the returned set.

Returns:

an unmodifiable set of keys in this property list where the key and its corresponding value are strings, including the keys in the default property list.

Since:

1.6

See Also:

`defaults`

list

```
public void list(PrintStream out)
```

Prints this property list out to the specified output stream. This method is useful for debugging.

Parameters:

`out` - an output stream.

Throws:

`ClassCastException` - if either a key or a value in this property list is not a string.

list

```
public void list(PrintWriter out)
```

Prints this property list out to the specified output stream. This method is useful for debugging.

Parameters:

`out` - an output stream.

Throws:

`ClassCastException` - if either a key or a value in this property list is not a string.

Since:

1.1

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.